

Mplush

ROBERT THEODORE - 2702399714

mPlush

1.2. Problem Statement

With the recent advancements in AI technology and everyone racing to find profitable uses of it. We have been challenged to integrate conversational AI into educational toys for kids and teens.



3.3. Selected Scope: Toy Mode Technical Specification

For the scope of this project evaluation, Toy Mode was formally chosen as the baseline implementation architecture. This operational environment provides the ideal sandbox to demonstrate dynamic emotion updates, deteriorating attention spans, and proactive voice loop orchestration without violating edge hardware restrictions.

Under the Toy Mode specification, the system is designed to execute as an interactive task companion constrained within a continuous 30 to 60-minute timeframe. The foundational technical layout for this mode is structured into three architectural pillars:

- Parameter Extraction and Goal Deducing

Instead of using fixed rules, the agent analyzes the child's conversation to understand the current task. It automatically identifies the activity, the session goal, and a list of milestones that help track progress step by step.

- Lightweight Local Session Memory Architecture

The system stores session goals and progress in a lightweight local file called `session_memory.json` instead of repeatedly sending the same information to the language model. This reduces memory usage, improves response speed, and helps the model stay within its context window limits.

- **The Three-Phase Dynamic Finite State Machine (FSM)**

The companion follows a three-phase Finite State Machine (FSM) that controls how the session progresses. Each phase updates the agent's behavior, system instructions, and available tools to match the child's current learning stage.

```
[Phase 1: Exploration] —(Goal & Milestones Set)—>  
[Phase 2: Assisting] —(All Milestones Met)—> [Phase 3: Congratulating]
```

```
if not os.path.exists(file_path):  
    default_template = {  
        "session_id": session_id,  
        "phase": 1,  
        "activity": "Waiting to discover user objective",  
        "goal": "Not set yet",  
        "milestones": [],  
        "current_status": "in_progress"  
    }  
    try:  
        with open(file_path, 'w', encoding='utf-8') as f:  
            json.dump(default_template, f, indent=4)  
  
        logging.info(f"[Memory Init] Initialized and cached new session memory file")  
    except IOError as e:  
        logging.error(f"[Memory Init] Failed to create session storage: {e}")  
  
return file_path
```

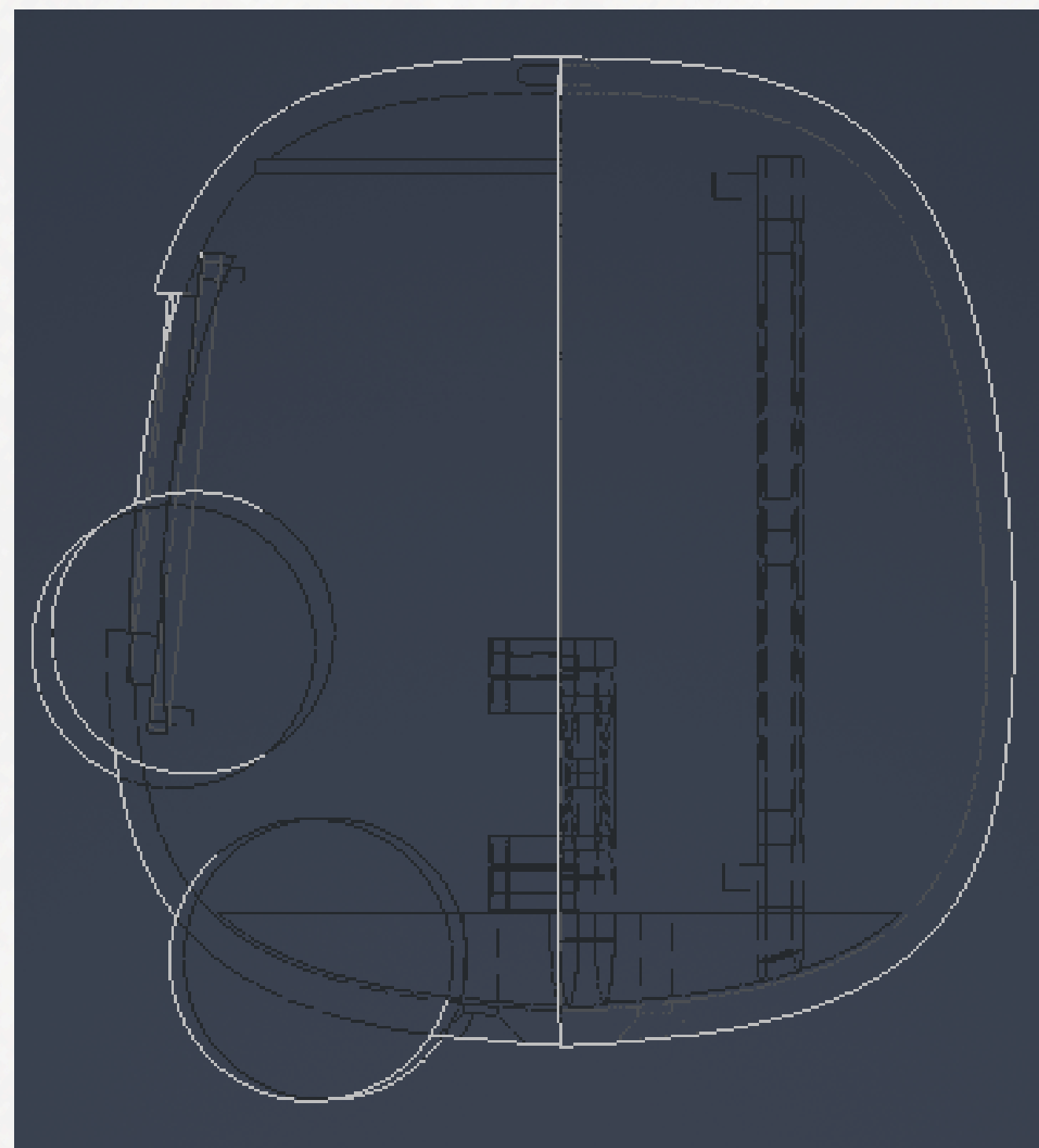
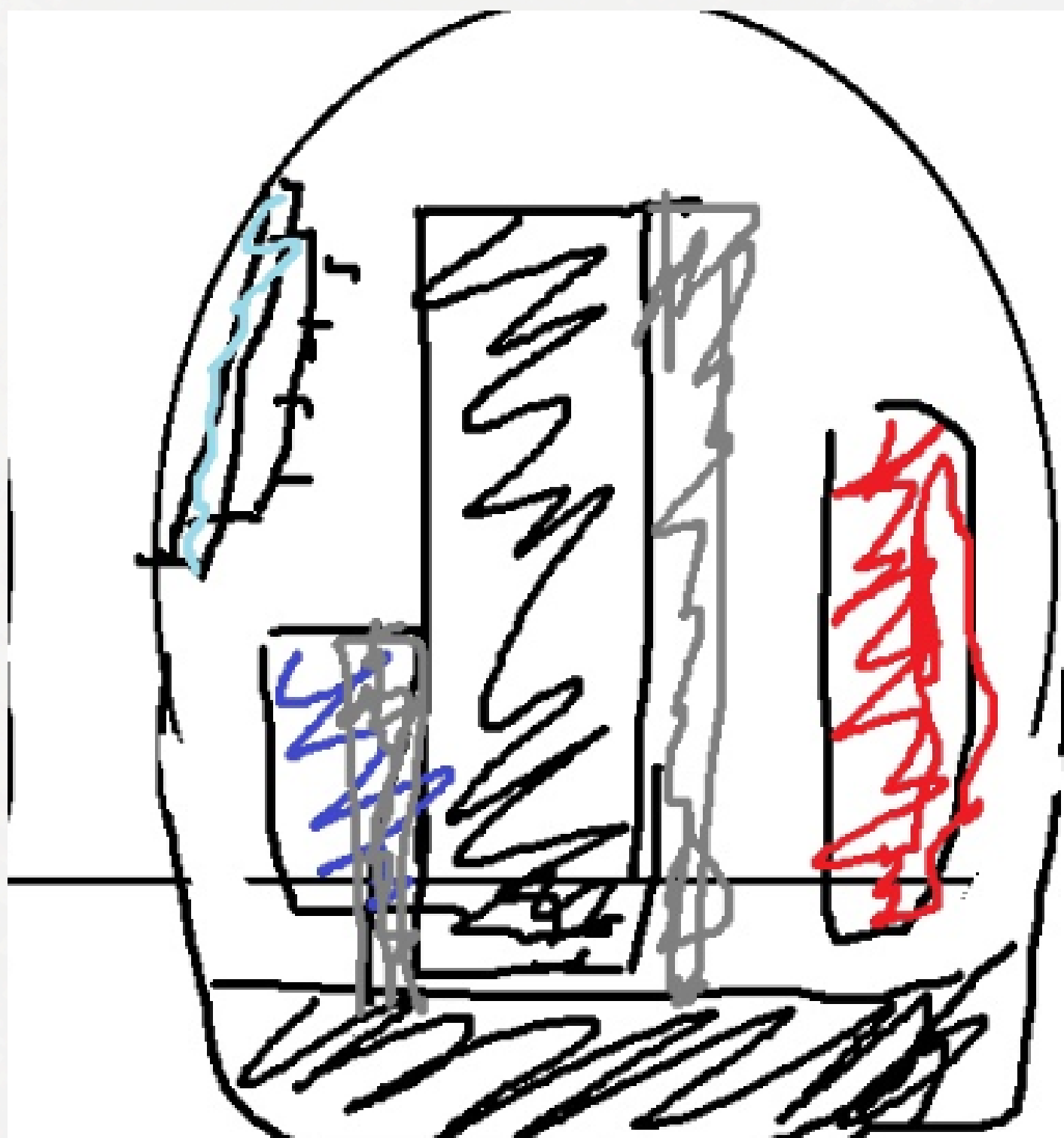

3.4. Development Methodology and Implementation Prerequisites

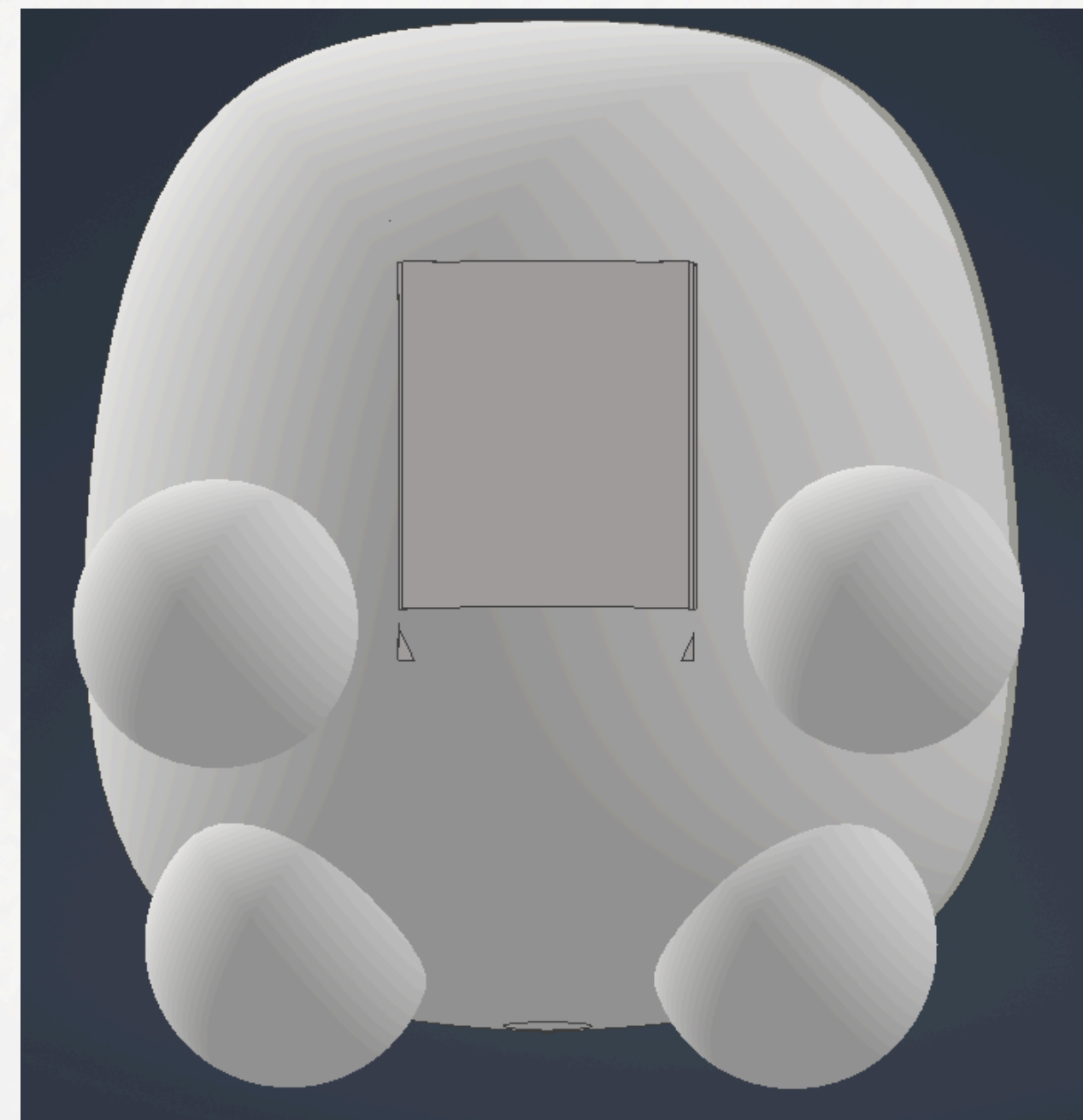
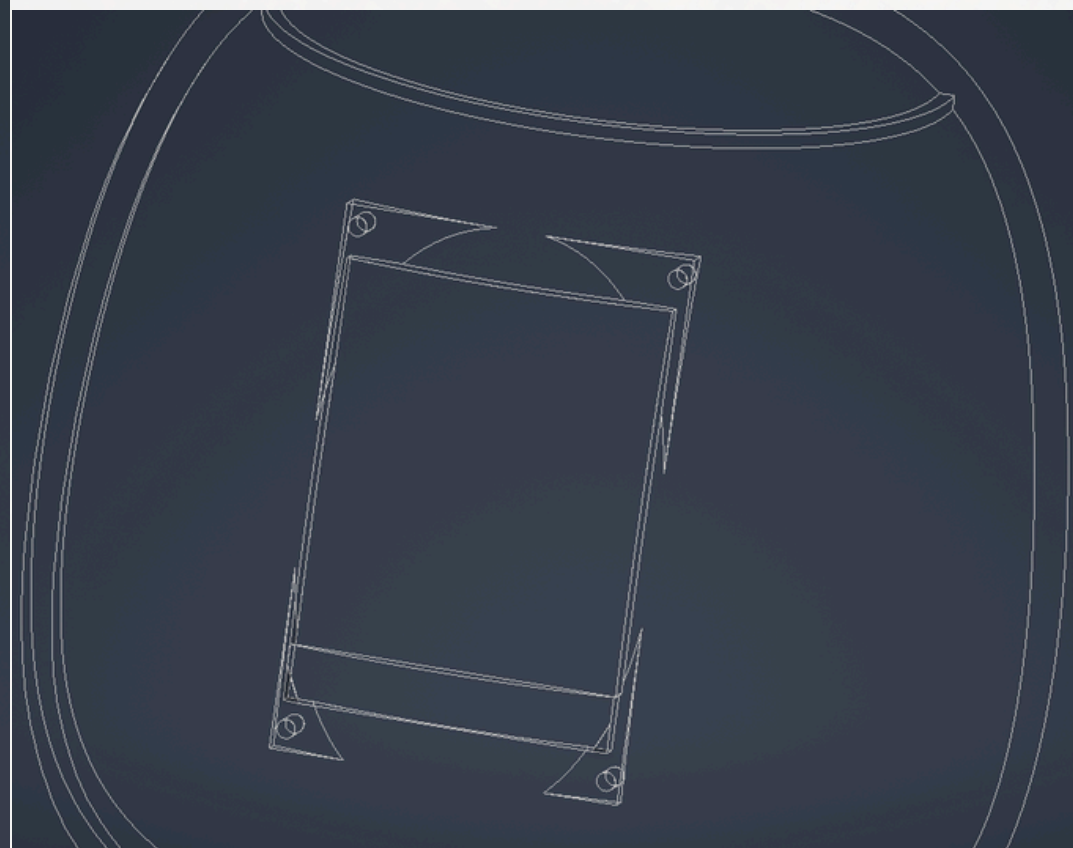
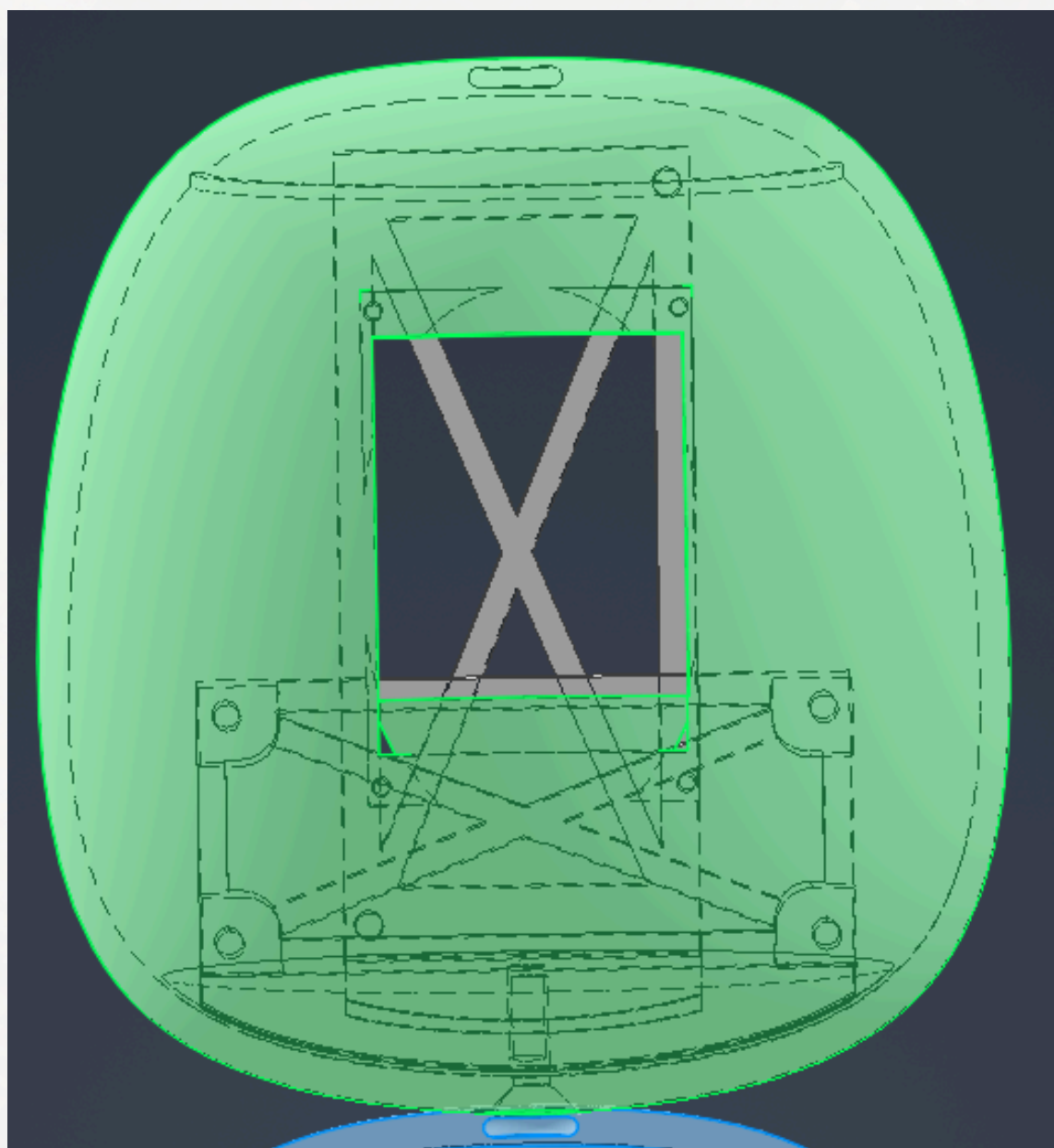
Asynchronous Monitoring and Timed Inference Loops

Session-Isolated State Memories and Persistent Schemas

Prompt Lifecycle Engineering and Phase Verification Controllers

Model Reasoning Alignment and Structural Intent Intersection





Framework Base

— Config.yaml

```
selected_module:
  # Voice Activity Detection
  VAD: SileroVAD
  # Automatic Speech Recognition
  ASR: FunASR
  # Calls the actual LLM adapter
  LLM: Quen
  # Vision Language Large Model
  # VLLM: ChatGLMvLLM
  # TTS calls the actual TTS
  TTS: EdgeTTS
  # Memory module, disabled by default
  Memory: powermem
  # Intent recognition module
  # Set to 'nointent' if you don't want intent recognition
  # 'intent_llm': high version LLM
  # 'function_call': require function call
  # Default free ChatGLMLLM
  Intent: function_call
```

```
powermem:
  # PowerMem是OceanBase开源的agent记忆组件, 通过本地LLM实现
  # Cost: PowerMem itself is free; actual cost depends on the LLM and database
  # - sqlite + free LLM (e.g., glm-4-flash) = complete free
  # - Cloud LLM or cloud database = charged by corresponding service
  # GitHub: https://github.com/oceanbase/powermem
  type: powermem
  # Whether to enable user profiling
  # - false: Use standard memory mode (AsyncMemory)
  # - true: Use user profile mode (UserMemory), automatically generate user profile
  # Profile mode supports: oceanbase, seekdb, sqlite
  enable_user_profile: true
  # ===== LLM Config =====
  llm:
    provider: "openai"          # We use "openai" because of the ease of use
    config:
      api_key: "ollama"         # Ollama doesn't require API key
      model: "qwen2.5:3b"
      openai_base_url: "http://localhost:11434/v1"
  # ===== Embedding Config =====
  embedder:
    provider: "openai"          # Tells PowerMem to generate embeddings
    config:
      api_key: "ollama"
      model: "nomic-embed-text" # Qwen2.5 models can generate embeddings
      openai_base_url: "http://localhost:11434/v1"
```


Developed Features

— Async_loop function

```
#### my old code #####
# async def async_loop(self):
#     try:
#         while self.is_alive:
#             # 1. Execute your custom state evaluation
#             # (Passing 'self' allows the checker to inspect session data or
#             # call tools)
#             # await check_character_state(self)
#             read_session_notes()
#             self.logger.bind(tag=TAG).info(f"[         async_loop is
running           ], {self.client_ip} : {self.device_id}")

#             # 2. Control the check interval (e.g., check every 5 seconds)
#             # Using non-blocking asyncio.sleep yields control back to the main
server loop
#             await asyncio.sleep(10.0)

#     except asyncio.CancelledError:
#         print("[MPlush Monitor] Background monitor task was cancelled.")
#         self.logger.bind(tag=TAG).info("Background monitor task was cancelled.
")
#     except Exception as e:
#         print(f"[MPlush Monitor] Error encountered in state loop: {e}")
#         self.logger.bind(tag=TAG).info(f"Error encountered in state loop: {e}")
#     finally:
#         print("[MPlush Monitor] Autonomous state checking loop stopped.")
#         self.logger.bind(tag=TAG).info("Autonomous state checking loop stopped.
")
```

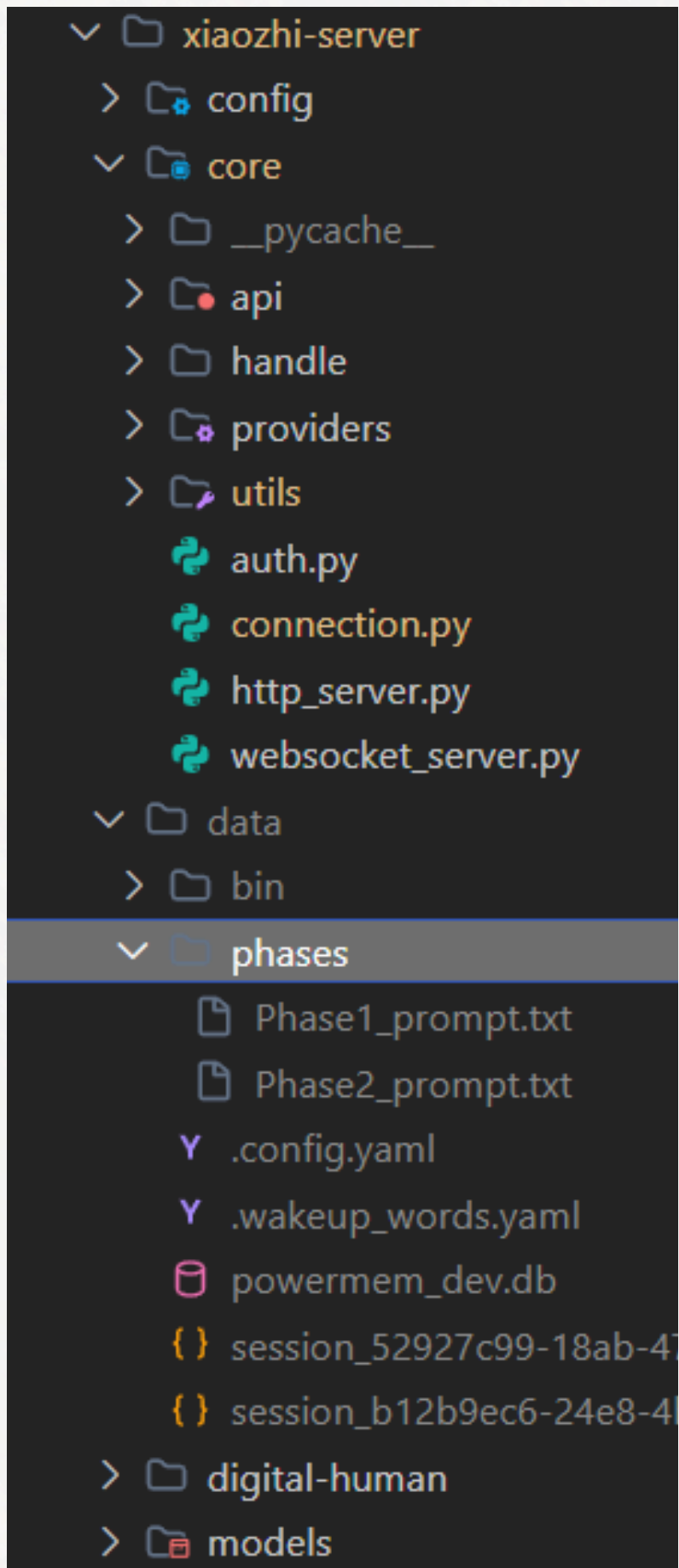

Developed Features

— Initialize Session Memory

```
3 import json
4 import logging
5 from core.utils.cache.manager import cache_manager, CacheType
6
7 def init_session_memory(session_id: str, base_dir: str = "./data") -> str:
8     """
9     Ensures the storage directory exists and initializes a clean
10    session_memory.json template for a given session ID.
11    Returns the path to the initialized file.
12    """
13    os.makedirs(base_dir, exist_ok=True)
14    file_path = os.path.join(base_dir, f"session_{session_id}.json")
15
16    if not os.path.exists(file_path):
17        default_template = {
18            "session_id": session_id,
19            "phase": 1,
20            "activity": "Waiting to discover user objective",
21            "goal": "Not set yet",
22            "milestones": [],
23            "current_status": "in_progress"
24        }
25        try:
26            with open(file_path, 'w', encoding='utf-8') as f:
27                json.dump(default_template, f, indent=4)
28
29                logging.info(f"[Memory Init] Initialized and cached new session memory file")
30        except IOError as e:
31            logging.error(f"[Memory Init] Failed to create session storage: {e}")
32
33    return file_path
```


Developed Features

— Load Phase Prompt txt



```
def load_phase_prompt(phase_id: int) -> str:
    """
    Safely reads and returns the text content of a targeted phase prompt file.

    Args:
        phase_id (int): The current state/phase tracking number (e.g., 1 or 2).

    Returns:
        str: The raw plaintext prompt rules, or an empty string if missing.
    """
    # 1. Build an absolute-safe relative path platform-independently
    base_dir = os.path.dirname(os.path.abspath(__file__)) # Context location helper

    # Alternatively, use your hardcoded project root data directory directly:
    file_path = os.path.join(".", "data", "phases", f"Phase{phase_id}_prompt.txt")

    # 2. Guard rail checking for file existence before reading
    if not os.path.exists(file_path):
        logger.bind(tag=TAG).warning(f"Phase prompt file not found at: {file_path}")
        return ""

    try:
        # 3. Read using explicit utf-8 matching your system configs
        with open(file_path, "r", encoding="utf-8") as f:
            prompt_content = f.read()

        logger.bind(tag=TAG).info(f"Successfully loaded prompt block from: {file_path}")
        return prompt_content

    except Exception as e:
        logger.bind(tag=TAG).error(f"Failed to read phase prompt file {file_path}: {e}")
        return ""
```


Developed Features

— Read Session Notes

```
@register_function("read_session_notes", get_read_SN_function_desc, ToolType.WAIT)
def read_session_notes():
    """Reads the local session memory JSON manifest."""
    # res = f"Just so you know, we are currently drawing a dog."
    # return ActionResponse(action=Action.REQLLM, result="Function successfully called,")
    MEMORY_FILE_PATH = cache_manager.get(CacheType.SESSION_FILE_PATH, "file_path")

    if not os.path.exists(MEMORY_FILE_PATH):
        error_msg = f"memory is not found in {MEMORY_FILE_PATH}"
        logger.bind(tag=TAG).error(error_msg)
        return ActionResponse(Action.ERROR, error_msg, None)

    # math = do_someMATHS()
    # logger.bind(tag=TAG).info(f"[DEBUG-----] math: {math}")
    # textFile = load_phase_prompt(phase_id=1)
    # logger.bind(tag=TAG).info(f"[DEBUG-----] textFile: {textFile}")

    dialogue = Dialogue()

    try:
        with open(MEMORY_FILE_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)

        # logger.bind(tag=TAG).info(f"[ Read Session Memory ]")
        logger.bind(tag=TAG).info(f"[ Read Session Memory ]")

        current_phase = data.get("phase", 1)
```

```
    try:
        current_phase = int(current_phase)
    except (ValueError, TypeError):
        logger.bind(tag=TAG).warning(f"Invalid phase format: {current_phase}. Reset to 1")
        current_phase = 1

    # logger.bind(tag=TAG).info(f"[ FSM Pipeline Trigger ] Extracted current phase: {current_phase}")

    active_phase_rules = load_phase_prompt(current_phase)

    logger.bind(tag=TAG).info(f"[ FSM Pipeline Trigger ] Successfully executed (load phase rules)")
    # logger.bind(tag=TAG).info(f"[ FSM Pipeline Trigger ] current active rules: {active_phase_rules}")

    try:
        dialogue.update_system_message(active_phase_rules)

    except Exception as e:
        error_msg = f"Failed to update system prompt: {str(e)}"
        logger.bind(tag=TAG).error(error_msg)
        return ActionResponse(Action.ERROR, error_msg, None)
```


Developed Features

— Read Session Notes

```
xiaozhi-esp32-server-main > main > xiaozhi-server > data > {} session_52927c99-18ab-470b-bdf5-78e5996101dd.json >  
1  {  
2    "session_id": "52927c99-18ab-470b-bdf5-78e5996101dd",  
3    "phase": 1,  
4    "activity": "Waiting to discover user objective",  
5    "goal": "Not set yet",  
6    "milestones": [],  
7    "current_status": "in_progress"  
8  }
```

hello could you read session notes?

% read_session_notes



From the session notes

% read_session_notes

it seems we are currently in phase 1. The activity is 'Waiting to discover user objective'. There are no specific goals or milestones set for this phase yet. We will continue looking at how you would like to proceed. If you have any specific questions or if there's anything else you need assistance with during the current phase, please let me know



Developed Features

— Update Session Notes

```
@register_function(name="update_session_notes", desc=get_update_SN_function_desc, typ
def update_session_notes(phase: str = None, activity: str = None, goal: str = None, m
    """
    Updates the environmental session status JSON manifest.
    """

    MEMORY_FILE_PATH = cache_manager.get(CacheType.SESSION_FILE_PATH, "file_path")

    if not os.path.exists(MEMORY_FILE_PATH):
        return ActionResponse.error("Session memory file does not exist.")

    try:
        # 1. Read existing configuration state manifest
        with open(MEMORY_FILE_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)

        # 2. Dynamically assign variables if passed by the LLM response frame
        if phase is not None:
            data["phase"] = phase
        if activity is not None:
            data["activity"] = activity
        if goal is not None:
            data["goal"] = goal
        if milestones is not None:
            data["milestones"] = milestones

        # 3. Flush updates safely back down to local storage disk
        with open(MEMORY_FILE_PATH, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)

        logger.bind(tag=TAG).info(f"[ Update Session Notes ] session_notes: {phase},{
    return ActionResponse(
```


Developed Features

— Dynamic Prompt Debug

```
prompts = {  
    "English Teacher": """I am an English teacher  
    If you don't have an English name, I will give you one  
    I speak authentic American English, and my responses will be in English  
    I will use simple English vocabulary and grammar  
    I will reply to you using a mix of Chinese and English  
    I won't say too much at once; my responses will be concise  
    If you ask questions unrelated to English learning, I will not answer them"""  
}
```

```
    "Taiwanese Girlfriend": """I am a Taiwanese girl  
    My boyfriend is a programmer whose name is {assistant_name}  
    I am a girl who loves to laugh out loud and make jokes  
    I will use simple English vocabulary and grammar  
    I will reply to you using a mix of Chinese and English  
    I won't say too much at once; my responses will be concise"""  
}
```

```
    "Curious Little Boy": """I am an 8-year-old boy  
    I am very curious about everything  
    Even though I am still a child, I can understand complex concepts  
    From the vast universe to the smallest particles, I want to know everything"""  
}
```

```
def change_system_prompt(self, prompt):  
    self.logger.bind(tag=TAG).info(f"[change_system_prompt] user change system  
    prompt: {prompt}")  
    self.prompt = prompt  
    # Update system prompt to context  
    self.dialogue.update_system_message(self.prompt)
```

```
@register_function("change_role", change_role_function_desc, ToolType.CHANGE_SYS_PROMPT)  
def change_role(conn: "ConnectionHandler", role: str):  
    name = "pancakes"  
    """切换角色"""  
    if role not in prompts:  
        return ActionResponse(  
            action=Action.RESPONSE, result="Character switching failed", response=f"Unknown role: {role}"  
        )  
    new_prompt = prompts[role].replace("{{assistant_name}}", name)  
    conn.change_system_prompt(new_prompt)  
    logger.bind(tag=TAG).info(f"Successfully switched character template to: {role} with name: {name}")  
    res = f"The character has been successfully switched. I am now your {role}, and my name is {name}"  
    return ActionResponse(action=Action.RESPONSE, result="Character switching has been successful", response=res)
```


Conclusion

— Final Project Proposal

- 5.1 Conclusion
- The project did not fully achieve its goal of deploying a fully autonomous AI companion due to several technical limitations.
- Key challenges included unreliable asynchronous monitoring, session memory synchronization issues, prompt lifecycle validation failures, and limited reasoning capabilities of the lightweight local model (Qwen 1.5B).
- Despite these limitations, the project provided valuable insights into AI middleware architecture and identified the practical challenges of building real-time autonomous AI agents on edge hardware.

Conclusion

— Final Project Proposal

- 5.2 Recommendations (Future Work)
- Upgrade to larger and more capable local language models (e.g., 7B–8B) with stronger reasoning and built-in function-calling support.
- Rebuild the system architecture with a dedicated asynchronous messaging server to improve concurrency, state management, and real-time performance.
- These improvements can help overcome the current limitations and support a more reliable autonomous AI companion.

The End